CLASE 33

Django Forms

Formularios que se validan solos

- Crear formularios con Django Forms
- ModelForm: formularios desde modelos
- Validar datos del usuario
- Renderizar formularios en templates
- CRUD completo: Crear, Leer, Editar, Eliminar
- Mensajes de feedback al usuario

Completar el CRUD del Dashboard E-commerce

Por que usar Django Forms?

HTML puro (tedioso y peligroso)

- X Escribir cada <input> manualmente
- X Validar con JavaScript Y en servidor
- X Mostrar errores manualmente
- X Proteger contra CSRF manualmente
- X Re-llenar datos si hay error
- X Manejar archivos, fechas, relaciones...

Django Forms (automatico)

- ✓ Genera HTML automaticamente
- ✓ Valida en el servidor
- ✓ Muestra errores junto al campo
- ✓ Proteccion CSRF incluida
- ✓ Re-llena datos si hay error
- ✓ Maneja archivos, fechas, relaciones

Django Forms = Seguridad + Productividad

Django Form basico

Un Form es una clase Python que define campos

- Hereda de forms.Form

Cada atributo es un campo del formulario
Los widgets controlan como se renderiza en HTML

Widgets controlan el HTML

- TextInput, Textarea, Select

RadioSelect, CheckboxInput
Se pueden agregar clases CSS (Bootstrap)

Tipos de campos comunes

`CharField`	Input de texto
`EmailField`	Email con validacion
`IntegerField`	Numero entero
`DecimalField`	Numero decimal
`BooleanField`	Checkbox
`ChoiceField`	Select / dropdown
`DateField`	Fecha
`FileField`	Subir archivo

Form = Definicion de campos + Validacion. Lo crearemos en VS Code.

ModelForm - Formularios automaticos

En lugar de definir cada campo, usa el modelo existente

Django genera el formulario completo basandose en los campos del modelo. Solo necesitas indicar que modelo y que campos incluir.

Campo del modelo -> HTML generado

`CharField`	-> <input type="text">
`TextField`	-> <textarea>
`DecimalField`	-> <input type="number">
`BooleanField`	-> <input type="checkbox">
`ForeignKey`	-> <select> con opciones
`ManyToMany`	-> Checkboxes / multi-select
`DateField`	-> <input type="date">

Opciones de class Meta:

`model`	El modelo a usar
`fields`	Lista de campos a incluir
`exclude`	Campos a excluir
`labels`	Nombres personalizados
`widgets`	Como renderizar cada campo
`help_texts`	Texto de ayuda para campos

ModelForm = El modelo define el formulario. Una linea y listo!

Conectar Form con View

El flujo GET / POST

GET: Usuario visita la pagina -> Formulario VACIO

POST: Usuario envia el formulario
-> Valido: guardar y redirigir
-> Invalido: mostrar errores

Metodos clave

`form.is_valid()`	Ejecuta todas las validaciones
`form.save()`	Guarda en BD (solo ModelForm)
`form.cleaned_data`	Datos limpios y validados
`form.errors`	Diccionario con errores

Patron de la view (siempre igual):

POST? -> Crear form con datos -> Validar -> Guardar -> Redirect

Siempre validar antes de guardar. El codigo lo veremos en VS Code.

Validacion automatica

Django valida basandose en el modelo:

Campo del modelo	Validacion automatica
`CharField(max_length=100)`	Maximo 100 caracteres
`EmailField()`	Formato email valido
`DecimalField(max_digits=10)`	Numero valido, decimales correctos
`IntegerField()`	Entero valido
`ForeignKey()`	La opcion debe existir en BD
`unique=True`	No se permiten duplicados

Errores comunes que Django detecta:

- "Este campo es requerido"

"Asegurese de que este valor sea menor o igual a X"
"Introduzca un numero" | "Introduzca una direccion de correo valida"

No necesitas escribir validaciones basicas. Django las hace por ti!

Validacion personalizada

clean_<campo>() - Un campo

- Valida UN campo especifico

Puede transformar el valor (capitalizar, limpiar)
Usa raise ValidationError si es invalido
SIEMPRE retorna el valor al final

clean() - Multiples campos

- Valida COMBINACIONES de campos

Ejemplo: precio alto requiere stock minimo
Llama a super().clean() primero
Retorna cleaned_data al final

Orden de validacion:

Validaciones del modelo -> clean_campo() por cada campo -> clean() validacion cruzada

SIEMPRE retornar el valor en clean_<campo>(). Si no, el campo queda vacio!

EJERCICIO: Crear ClienteForm

Campos a incluir:

- nombre -> TextInput con 'form-control'

email -> EmailInput con placeholder
telefono -> TextInput (campo opcional)

Validaciones personalizadas:

clean_telefono():
Al menos 8 digitos, quitar espacios/guiones
clean_nombre():
No puede ser solo numeros, capitalizar

Pasos:

1. Crear dashboard_app/forms.py
2. Crear ClienteForm con ModelForm
3. Agregar widgets con clases Bootstrap
4. Agregar validaciones personalizadas

8 min

Renderizar formularios en templates

Automatico (rapido)

```
{{ form.as_p }}
{{ form.as_table }}
{{ form.as_ul }}
{{ form.as_div }}
```

Manual (mas control)

```
{{ form.nombre.label_tag }}
{{ form.nombre }}
{{ form.nombre.errors }}
{{ form.nombre.help_text }}
```

Loop (recomendado)

```
{% for field in form %}
  {{ field.label_tag }}
  {{ field }}
  {{ field.errors }}
{% endfor %}
```

Puntos clave del template:

- {% csrf_token %} es OBLIGATORIO dentro del <form> (proteccion contra ataques)

novalidate en el <form> desactiva validacion del navegador (usamos la de Django)
form.non_field_errors muestra errores del metodo clean() (validacion cruzada)
field.field.required nos dice si el campo es requerido (para mostrar asterisco)

SIEMPRE incluir {% csrf_token %} en el <form>. Sin el, Django rechaza el POST!

Agregar URL para el formulario

Nueva URL en productos/urls.py:

```
path('nuevo/', views.producto_crear, name='crear')
```

Flujo completo:

1. Usuario hace clic en "Nuevo Producto"
2. GET /productos/nuevo/ -> Formulario vacio
3. Usuario llena los campos y envia
4. POST /productos/nuevo/ -> Django valida
5. Si valido -> Guardar -> Redirect a lista
6. Si invalido -> Mostrar formulario con errores

Probemos el formulario en el navegador!

BREAK - 10 MINUTOS

Hasta ahora:

- Form basico y ModelForm

Validacion automatica y personalizada
Renderizar en templates
Crear producto funcionando

Volvemos con:

- Editar y eliminar (CRUD completo)

Mensajes flash
CRUD de Cliente

CRUD: Update - Editar registros

La diferencia clave: instance

CREAR:
`ProductoForm()`

EDITAR:
`ProductoForm(instance=producto)`

instance hace dos cosas:

En GET: Pre-llena el formulario con los datos actuales del producto

En POST: ACTUALIZA ese producto en lugar de crear uno nuevo

- El mismo template funciona para crear y editar

Solo cambiamos el titulo que pasamos desde la view
Necesitamos una nueva URL: /<id>/editar/

instance pre-llena el formulario Y actualiza en lugar de crear nuevo

CRUD: Delete - Eliminar registros

Regla de seguridad:

- NUNCA eliminar con GET (los enlaces pueden pre-cargarse por navegadores y bots)

GET muestra pagina de confirmacion -> POST ejecuta la eliminacion

Patron de la view:

- 1. Buscar el objeto con get_object_or_404

2. Si POST: guardar nombre, eliminar, redirect
3. Si GET: mostrar pagina de confirmacion
4. Agregar mensaje de exito

Template de confirmacion:

- Card con borde rojo para alertar

Pregunta clara: "Estas seguro?"
Boton rojo "Si, eliminar"
Boton gris "Cancelar"
Un <form> con POST y csrf_token

Siempre pedir confirmacion antes de eliminar. Imaginen si Google indexa /eliminar/!

CRUD completo - URLs de Productos

Patron CRUD:

Operacion	Metodo	URL	Vista
Create	GET / POST	`/nuevo/`	`producto_crear`
Read (lista)	GET	`/`	`lista_productos`
Read (detalle)	GET	`/<id>/`	`detalle_producto`
Update	GET / POST	`/<id>/editar/`	`producto_editar`
Delete	GET / POST	`/<id>/eliminar/`	`producto_eliminar`

Este patron se repite para cada recurso:

Productos, Clientes, Ventas, Categorias... mismo patron, diferentes modelos. Una vez que lo aprenden, lo pueden aplicar a cualquier modelo.

5 URLs, 5 views, 4 templates = CRUD completo para cualquier recurso

EJERCICIO: CRUD completo de Cliente

Views a crear en dashboard_app/views.py:

- cliente_lista -> Listar todos
- cliente_crear -> Formulario crear
- cliente_detalle -> Ver un cliente
- cliente_editar -> Formulario editar
- cliente_eliminar -> Confirmacion y eliminar

URLs a agregar:

```
/clientes/
/clientes/nuevo/
/clientes/<id>/
/clientes/<id>/editar/
/clientes/<id>/eliminar/
```

Templates necesarios:

- cliente_lista.html

cliente_form.html (crear y editar)
cliente_detalle.html
cliente_confirmar_eliminar.html

15 min

Sigan el mismo patron de Productos

Messages Framework - Feedback al usuario

En la view:

`messages.success()`	Exito (verde)
`messages.error()`	Error (rojo)
`messages.warning()`	Advertencia (amarillo)
`messages.info()`	Info (azul)

En base.html (una sola vez):

- Agregar bloque de mensajes despues del nav

Loop sobre messages
Usar clases de Bootstrap (alert-success, etc.)
Boton X para cerrar la alerta

Caracteristicas de los mensajes:

- Se muestran UNA sola vez y desaparecen despues de mostrarse

Funcionan incluso despues de redirect (se guardan en la sesion)
Perfectos para confirmaciones: "Producto creado", "Cliente eliminado"
Hay que configurar MESSAGE_TAGS en settings.py para que error use 'danger' de Bootstrap

Siempre dar feedback al usuario despues de crear, editar o eliminar

Patron CRUD - Checklist

1. forms.py

- Crear ModelForm con campos y widgets
- Agregar validaciones personalizadas

2. views.py

- lista: QuerySet + template
- crear: Form vacio / Form(POST) + save
- detalle: get_object_or_404
- editar: Form(instance) / Form(POST,instance)
- eliminar: Confirmacion + delete()

3. urls.py

- /recurso/ (lista)
- /recurso/nuevo/ (crear)
- /recurso/<id>/ (detalle)
- /recurso/<id>/editar/ (editar)
- /recurso/<id>/eliminar/ (eliminar)

4. templates/

- lista.html (con boton Nuevo)
- form.html (crear y editar comparten)
- detalle.html (con botones editar/eliminar)
- confirmar_eliminar.html

Este patron se repite para cada recurso. Despues de 2-3 veces, sale automatico!

Resumen - Django Forms

Lo que logramos hoy:

- Form basico y ModelForm
- Validacion automatica desde el modelo
- Validacion personalizada (clean methods)
- Renderizar formularios en templates
- CRUD completo de Producto
- CRUD completo de Cliente
- Messages framework para feedback
- Patron reutilizable para cualquier recurso

PROXIMA CLASE: Auth + Deploy

(ULTIMA)

- Sistema de usuarios de Django
- Login y logout
- Registro de usuarios
- Proteger vistas (@login_required)
- Deploy a produccion

Proyecto Dashboard E-commerce:

Setup + URLs	Templates	Models	Forms/CRUD	Auth+Deploy